

# ASSIGNMENT NO.9: Data Retrieval, Functions, and Joins in MySQL

## Objective:

The objective of this practical is to help students understand how to combine results from multiple MySQL queries using set operations. Students will learn how to use UNION, INTERSECT, and EXCEPT, combine multiple query results, apply advanced filtering techniques, and solve analytical problems using MySQL queries.

**MySQL Note:** UNION is commonly supported in MySQL. INTERSECT and EXCEPT are supported in MySQL 8.0.31 and later versions. In current MySQL documentation, MySQL supports UNION, INTERSECT, and EXCEPT as set operations.

## 1. Introduction to Set Operations

Set operations are used to combine the results of two or more SELECT queries. These operations are useful when similar data is stored in different tables and we want to compare, combine, or analyze records.

For example, a college may have two admission sources:

| Admission Source   | Description                                          |
|--------------------|------------------------------------------------------|
| Online Enrollment  | Students who applied through online admission portal |
| Offline Enrollment | Students who applied directly at the college office  |

Using set operations, we can find:

| Requirement                            | Suitable Set Operation |
|----------------------------------------|------------------------|
| All students from both sources         | UNION                  |
| Students present in both sources       | INTERSECT              |
| Students present only in online source | EXCEPT                 |
| Combine filtered results               | UNION with WHERE       |
| Compare two datasets                   | INTERSECT or EXCEPT    |

## Important Rules for Set Operations

| Rule                   | Explanation                                                         |
|------------------------|---------------------------------------------------------------------|
| Same number of columns | Both SELECT queries should return equal number of columns           |
| Compatible data types  | Matching columns should have similar data types                     |
| Column names           | Final output usually takes column names from the first SELECT query |
| Duplicate handling     | UNION, INTERSECT, and EXCEPT remove duplicate rows by default       |
| Include duplicates     | Use ALL when duplicate rows are required                            |

## Creating Sample Tables and Inserting Records

For this practical, we will use two tables: OnlineEnrollment and OfflineEnrollment.

### OnlineEnrollment Table Structure

| Column Name | Data Type   | Constraint  |
|-------------|-------------|-------------|
| StudentID   | INT         | PRIMARY KEY |
| StudentName | VARCHAR(50) | NOT NULL    |
| Course      | VARCHAR(30) |             |
| City        | VARCHAR(30) |             |
| Marks       | INT         |             |

### OfflineEnrollment Table Structure

| Column Name | Data Type   | Constraint  |
|-------------|-------------|-------------|
| StudentID   | INT         | PRIMARY KEY |
| StudentName | VARCHAR(50) | NOT NULL    |
| Course      | VARCHAR(30) |             |
| City        | VARCHAR(30) |             |
| Marks       | INT         |             |

### MySQL Code to Create Database and Tables

```
CREATE DATABASE CollegeDB;
```

```
USE CollegeDB;
```

```
CREATE TABLE OnlineEnrollment ( StudentID INT PRIMARY KEY, StudentName  
VARCHAR(50) NOT NULL, Course VARCHAR(30), City VARCHAR(30), Marks INT );  
CREATE TABLE OfflineEnrollment ( StudentID INT PRIMARY KEY, StudentName  
VARCHAR(50) NOT NULL, Course VARCHAR(30), City VARCHAR(30), Marks INT );
```

### MySQL Code to Insert Records

```
INSERT INTO OnlineEnrollment  
(StudentID, StudentName, Course, City, Marks)  
VALUES  
(1, 'Rahul Sharma', 'BSc IT', 'Mumbai', 78),  
(2, 'Priya Patil', 'BCA', 'Pune', 85),  
(3, 'Aman Verma', 'BSc CS', 'Nashik', 67),  
(4, 'Sneha Joshi', 'BSc IT', 'Mumbai', 91),  
(5, 'Karan Mehta', 'BCA', 'Thane', 74),  
(6, 'Neha Shah', 'BSc IT', 'Pune', 88);
```

```
INSERT INTO OfflineEnrollment  
(StudentID, StudentName, Course, City, Marks)  
VALUES  
(4, 'Sneha Joshi', 'BSc IT', 'Mumbai', 91),  
(5, 'Karan Mehta', 'BCA', 'Thane', 74),  
(7, 'Rohan Desai', 'BSc CS', 'Mumbai', 69),  
(8, 'Meera Kulkarni', 'BCA', 'Nashik', 82),
```

```
(9, 'Arjun Nair', 'BSc IT', 'Thane', 59),  
(10, 'Divya More', 'BCA', 'Mumbai', 93);
```

#### OnlineEnrollment Table

| StudentID | StudentName  | Course | City   | Marks |
|-----------|--------------|--------|--------|-------|
| 1         | Rahul Sharma | BSc IT | Mumbai | 78    |
| 2         | Priya Patil  | BCA    | Pune   | 85    |
| 3         | Aman Verma   | BSc CS | Nashik | 67    |
| 4         | Sneha Joshi  | BSc IT | Mumbai | 91    |
| 5         | Karan Mehta  | BCA    | Thane  | 74    |
| 6         | Neha Shah    | BSc IT | Pune   | 88    |

#### OfflineEnrollment Table

| StudentID | StudentName    | Course | City   | Marks |
|-----------|----------------|--------|--------|-------|
| 4         | Sneha Joshi    | BSc IT | Mumbai | 91    |
| 5         | Karan Mehta    | BCA    | Thane  | 74    |
| 7         | Rohan Desai    | BSc CS | Mumbai | 69    |
| 8         | Meera Kulkarni | BCA    | Nashik | 82    |
| 9         | Arjun Nair     | BSc IT | Thane  | 59    |
| 10        | Divya More     | BCA    | Mumbai | 93    |

### Using UNION to Combine Query Results

The UNION operator is used to combine the result of two or more SELECT queries. It removes duplicate records by default.

#### Syntax

```
SELECT column1, column2 FROM table1  
UNION  
SELECT column1, column2 FROM table2;
```

#### Example 1: Display All Students from Online and Offline Enrollment

```
SELECT StudentID, StudentName, Course, City, Marks  
FROM OnlineEnrollment
```

#### UNION

```
SELECT StudentID, StudentName, Course, City, Marks  
FROM OfflineEnrollment;
```

#### Output

| StudentID | StudentName  | Course | City   | Marks |
|-----------|--------------|--------|--------|-------|
| 1         | Rahul Sharma | BSc IT | Mumbai | 78    |
| 2         | Priya Patil  | BCA    | Pune   | 85    |
| 3         | Aman Verma   | BSc CS | Nashik | 67    |
| 4         | Sneha Joshi  | BSc IT | Mumbai | 91    |

|    |                |        |        |    |
|----|----------------|--------|--------|----|
| 5  | Karan Mehta    | BCA    | Thane  | 74 |
| 6  | Neha Shah      | BSc IT | Pune   | 88 |
| 7  | Rohan Desai    | BSc CS | Mumbai | 69 |
| 8  | Meera Kulkarni | BCA    | Nashik | 82 |
| 9  | Arjun Nair     | BSc IT | Thane  | 59 |
| 10 | Divya More     | BCA    | Mumbai | 93 |

### Explanation

| SQL Part          | Purpose                                             |
|-------------------|-----------------------------------------------------|
| First SELECT      | Gets records from OnlineEnrollment                  |
| Second SELECT     | Gets records from OfflineEnrollment                 |
| UNION             | Combines both results and removes duplicate records |
| Duplicate records | Sneha Joshi and Karan Mehta appear only once        |

### UNION vs UNION ALL

| Operator  | Meaning                                 |
|-----------|-----------------------------------------|
| UNION     | Combines results and removes duplicates |
| UNION ALL | Combines results and keeps duplicates   |

### Example 2: Use UNION ALL

```
SELECT StudentID, StudentName, Course FROM OnlineEnrollment
UNION ALL
SELECT StudentID, StudentName, Course FROM OfflineEnrollment;
---
```

## 4. Using INTERSECT to Find Common Records

The 'INTERSECT' operator is used to display records that are common in both query results.

It is useful when we want to find matching records between two datasets.

### Syntax

```
SELECT column1, column2
FROM table1
```

```
INTERSECT
```

```
SELECT column1, column2
FROM table2;
```

### Example: Find Students Present in Both Online and Offline Enrollment

```
SELECT StudentID, StudentName, Course, City, Marks FROM OnlineEnrollment
INTERSECT
SELECT StudentID, StudentName, Course, City, Marks FROM OfflineEnrollment;
```

### Output

| StudentID | StudentName | Course | City | Marks |
|-----------|-------------|--------|------|-------|
| ---       | ---         | ---    | ---  | ---   |

|   |             |        |        |    |
|---|-------------|--------|--------|----|
| 4 | Sneha Joshi | BSc IT | Mumbai | 91 |
| 5 | Karan Mehta | BCA    | Thane  | 74 |

## Explanation

| Point           | Description                                                     |
|-----------------|-----------------------------------------------------------------|
| `INTERSECT`     | Finds records common to both query results                      |
| Common students | Sneha Joshi and Karan Mehta                                     |
| Use case        | Useful for identifying duplicate admissions or repeated records |

## MySQL Alternative for Older Versions

If `INTERSECT` is not available in the MySQL version being used, the same result can be achieved using `INNER JOIN`.

```
SELECT O.StudentID, O.StudentName, O.Course, O.City, O.Marks
FROM OnlineEnrollment O
INNER JOIN OfflineEnrollment F
ON O.StudentID = F.StudentID;
```

## Alternative Output

| StudentID | StudentName | Course | City   | Marks |
|-----------|-------------|--------|--------|-------|
| 4         | Sneha Joshi | BSc IT | Mumbai | 91    |
| 5         | Karan Mehta | BCA    | Thane  | 74    |

## Using EXCEPT to Find Difference Between Results

The EXCEPT operator is used to display records from the first query that are not present in the second query.

It is useful when we want to find records available in one dataset but missing in another dataset.

### Syntax

```
SELECT column1, column2 FROM table1
EXCEPT
SELECT column1, column2 FROM table2;
```

## Example: Find Students Only in Online Enrollment

```
SELECT StudentID, StudentName, Course, City, Marks
FROM OnlineEnrollment
```

## EXCEPT

```
SELECT StudentID, StudentName, Course, City, Marks
FROM OfflineEnrollment;
```

### Output

| StudentID | StudentName  | Course | City   | Marks |
|-----------|--------------|--------|--------|-------|
| 1         | Rahul Sharma | BSc IT | Mumbai | 78    |
| 2         | Priya Patil  | BCA    | Pune   | 85    |
| 3         | Aman Verma   | BSc CS | Nashik | 67    |
| 6         | Neha Shah    | BSc IT | Pune   | 88    |

### Explanation

| Point        | Description                                          |
|--------------|------------------------------------------------------|
| First query  | Gets records from OnlineEnrollment                   |
| Second query | Gets records from OfflineEnrollment                  |
| EXCEPT       | Removes records that are present in the second query |
| Final output | Shows only online-only students                      |

### MySQL Alternative for Older Versions

If EXCEPT is not available in the MySQL version being used, the same result can be achieved using LEFT JOIN with IS NULL.

```
SELECT O.StudentID, O.StudentName, O.Course, O.City, O.Marks FROM OnlineEnrollment O LEFT JOIN OfflineEnrollment F ON O.StudentID = F.StudentID WHERE F.StudentID IS NULL;
```

### Alternative Output

| StudentID | StudentName  | Course | City   | Marks |
|-----------|--------------|--------|--------|-------|
| 1         | Rahul Sharma | BSc IT | Mumbai | 78    |
| 2         | Priya Patil  | BCA    | Pune   | 85    |
| 3         | Aman Verma   | BSc CS | Nashik | 67    |
| 6         | Neha Shah    | BSc IT | Pune   | 88    |

### Combining Multiple Query Results with Filtering

Set operations can be combined with WHERE, ORDER BY, and selected columns to solve advanced filtering problems.

#### Example 1: Combine BCA Students from Both Tables

```
SELECT StudentID, StudentName, Course, City, Marks FROM OnlineEnrollment
WHERE Course = 'BCA'
UNION
SELECT StudentID, StudentName, Course, City, Marks FROM OfflineEnrollment
WHERE Course = 'BCA';
```

### Output

| StudentID | StudentName | Course | City | Marks |
|-----------|-------------|--------|------|-------|
|-----------|-------------|--------|------|-------|

|    |                |     |        |    |
|----|----------------|-----|--------|----|
| 2  | Priya Patil    | BCA | Pune   | 85 |
| 5  | Karan Mehta    | BCA | Thane  | 74 |
| 8  | Meera Kulkarni | BCA | Nashik | 82 |
| 10 | Divya More     | BCA | Mumbai | 93 |

## Example 2: Combine High-Scoring Students from Both Tables

```
SELECT StudentID, StudentName, Course, Marks
FROM OnlineEnrollment
WHERE Marks >= 85
```

### UNION

```
SELECT StudentID, StudentName, Course, Marks
FROM OfflineEnrollment
WHERE Marks >= 85
```

ORDER BY Marks DESC;

### Output

| StudentID | StudentName | Course | Marks |
|-----------|-------------|--------|-------|
| 10        | Divya More  | BCA    | 93    |
| 4         | Sneha Joshi | BSc IT | 91    |
| 6         | Neha Shah   | BSc IT | 88    |
| 2         | Priya Patil | BCA    | 85    |

## Example 3: Find Mumbai Students from Both Tables

```
SELECT StudentID, StudentName, City FROM OnlineEnrollment WHERE City = 'Mumbai'
```

### UNION

```
SELECT StudentID, StudentName, City FROM OfflineEnrollment WHERE City =
'Mumbai';
```

### Output

| StudentID | StudentName  | City   |
|-----------|--------------|--------|
| 1         | Rahul Sharma | Mumbai |
| 4         | Sneha Joshi  | Mumbai |
| 7         | Rohan Desai  | Mumbai |
| 10        | Divya More   | Mumbai |

## Explanation

| Technique | Purpose |
|-----------|---------|
| ---       |         |

|                             |                                           |
|-----------------------------|-------------------------------------------|
| `WHERE` with set operations | Filters records before combining          |
| `ORDER BY`                  | Sorts final combined result               |
| Selected columns            | Displays only required fields             |
| `UNION`                     | Removes duplicate rows in combined output |

## 7. Solving Analytical Problems using Set Operations

Set operations help in solving analytical problems where data must be compared across different sources. They are useful in admission systems, sales reports, attendance systems, placement records, and customer databases.

### Analytical Problem Examples

| Scenario                  | Requirement                                       | Suitable Query          |
|---------------------------|---------------------------------------------------|-------------------------|
| Admission analysis        | Find all students from online and offline sources | `UNION`                 |
| Duplicate admission check | Find students appearing in both sources           | `INTERSECT`             |
| Online-only applications  | Find students only in online enrollment           | `EXCEPT`                |
| Offline-only applications | Find students only in offline enrollment          | `EXCEPT`                |
| Course demand analysis    | Combine BCA or BSc IT students from both sources  | `UNION` with `WHERE`    |
| Merit list preparation    | Combine high-scoring students from both sources   | `UNION` with `ORDER BY` |

### Example 1: Count Total Unique Students from Both Sources

```
SELECT COUNT(*) AS TotalUniqueStudents
```

```
FROM (
    SELECT StudentID, StudentName
    FROM OnlineEnrollment
```

```
    UNION
```

```
    SELECT StudentID, StudentName
    FROM OfflineEnrollment
```

```
) AS CombinedStudents;
```

#### Output

| TotalUniqueStudents |
|---------------------|
| 10                  |

### Example 2: Count Common Students from Both Sources

```
SELECT COUNT(*) AS CommonStudents FROM ( SELECT StudentID, StudentName FROM
```

```
OnlineEnrollment
INTERSECT

SELECT StudentID, StudentName
FROM OfflineEnrollment
) AS CommonData;
```

Output

| CommonStudents |     |   |
|----------------|-----|---|
|                | --- | : |
| 2              |     |   |

Example 3: Count Online-Only Students

```
SELECT COUNT(*) AS OnlineOnlyStudents
FROM (
    SELECT StudentID, StudentName
    FROM OnlineEnrollment

    EXCEPT

    SELECT StudentID, StudentName
    FROM OfflineEnrollment
) AS OnlineOnlyData;
```

Output

| OnlineOnlyStudents |
|--------------------|
| 4                  |

Final Summary Table

| Concept                 | MySQL Command               | Purpose                                                 |
|-------------------------|-----------------------------|---------------------------------------------------------|
| Combine results         | UNION                       | Combines records and removes duplicates                 |
| Keep duplicates         | UNION ALL                   | Combines records and keeps duplicates                   |
| Find common records     | INTERSECT                   | Displays records present in both queries                |
| Find different records  | EXCEPT                      | Displays records from first query not present in second |
| Filter before combining | WHERE                       | Applies conditions before set operation                 |
| Sort final result       | ORDER BY                    | Arranges combined output                                |
| Analyze combined data   | Subquery with set operation | Counts or summarizes set results                        |
| Older MySQL alternative | JOIN, LEFT JOIN, NOT EXISTS | Used when INTERSECT or EXCEPT is unavailable            |

## Questions:

**Q1) Create a MySQL database named CollegeDB and create two tables: OnlineEnrollment and OfflineEnrollment.**

```
CREATE DATABASE CollegeDB;
```

```
USE CollegeDB;
```

```
CREATE TABLE OnlineEnrollment (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50) NOT NULL,  
    Course VARCHAR(30),  
    City VARCHAR(30),  
    Marks INT  
);
```

```
CREATE TABLE OfflineEnrollment (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50) NOT NULL,  
    Course VARCHAR(30),  
    City VARCHAR(30),  
    Marks INT  
);
```

```
INSERT INTO OnlineEnrollment VALUES  
(1,'Rahul Sharma','BSc IT','Mumbai',78),  
(2,'Priya Patil','BCA','Pune',85),  
(3,'Aman Verma','BSc CS','Nashik',67),  
(4,'Sneha Joshi','BSc IT','Mumbai',91),  
(5,'Karan Mehta','BCA','Thane',74),  
(6,'Neha Shah','BSc IT','Pune',88);
```

```
INSERT INTO OfflineEnrollment VALUES  
(4,'Sneha Joshi','BSc IT','Mumbai',91),
```

(5,'Karan Mehta','BCA','Thane',74),  
 (7,'Rohan Desai','BSc CS','Mumbai',69),  
 (8,'Meera Kulkarni','BCA','Nashik',82),  
 (9,'Arjun Nair','BSc IT','Thane',59),  
 (10,'Divya More','BCA','Mumbai',93);

**OnlineEnrollment Table**

| Column Name | Data Type   | Constraint  |
|-------------|-------------|-------------|
| StudentID   | INT         | PRIMARY KEY |
| StudentName | VARCHAR(50) | NOT NULL    |
| Course      | VARCHAR(30) |             |
| City        | VARCHAR(30) |             |
| Marks       | INT         |             |

**OfflineEnrollment Table**

| Column Name | Data Type   | Constraint  |
|-------------|-------------|-------------|
| StudentID   | INT         | PRIMARY KEY |
| StudentName | VARCHAR(50) | NOT NULL    |
| Course      | VARCHAR(30) |             |
| Column Name | Data Type   | Constraint  |
| City        | VARCHAR(30) |             |
| Marks       | INT         |             |

Insert at least 6 records into each table. Some records should be common in both tables.

**2) Write a MySQL query using UNION to display all unique students from both OnlineEnrollment and OfflineEnrollment.**

SELECT StudentID, StudentName, Course, City, Marks

FROM OnlineEnrollment

UNION

SELECT StudentID, StudentName, Course, City, Marks

FROM OfflineEnrollment;

The screenshot shows the phpMyAdmin interface for a database named 'collegedb'. The selected table is 'OnlineEnrollment'. A SQL query is entered in the query box:

```
SELECT StudentID, StudentName, Course, City, Marks FROM OnlineEnrollment UNION SELECT StudentID, StudentName, Course, City, Marks FROM OfflineEnrollment;
```

The results show 4 rows from the 'OnlineEnrollment' table:

| StudentID | StudentName | Course | City   | Marks |
|-----------|-------------|--------|--------|-------|
| 1         | Aarav       | BCA    | Pune   | 85    |
| 2         | Priya       | BSc IT | Mumbai | 78    |
| 3         | Rohan       | BSc CS | Delhi  | 92    |
| 4         | Neha        | BCA    | Pune   | 88    |

The output should contain:

| Column      |
|-------------|
| StudentID   |
| StudentName |
| Course      |
| City        |
| Marks       |

**Q3) Write a MySQL query using UNION ALL to combine records from both tables while keeping duplicate records.**

```
SELECT StudentID, StudentName, Course, City, Marks
```

```
FROM OnlineEnrollment
```

```
UNION ALL
```

```
SELECT StudentID, StudentName, Course, City, Marks
```

```
FROM OfflineEnrollment;
```

The screenshot shows the phpMyAdmin interface for a database named 'collegedb'. The table 'OnlineEnrollment' is selected. A SQL query is entered in the query box:

```
SELECT StudentID, StudentName, Course, City, Marks FROM OnlineEnrollment UNION ALL OfflineEnrollment;
```

The query execution shows 4 rows of data. The results are displayed in a table:

| StudentID | StudentName | Course | City   | Marks |
|-----------|-------------|--------|--------|-------|
| 1         | Aarav       | BCA    | Pune   | 85    |
| 2         | Priya       | BSc IT | Mumbai | 78    |
| 3         | Rohan       | BSc CS | Delhi  | 92    |
| 4         | Neha        | BCA    | Pune   | 88    |

Also write one line explaining the difference between UNION and UNION ALL.

**Q4) Write a MySQL query using INTERSECT to display students who are present in both online and offline enrollment tables.**

```
SELECT StudentID, StudentName, Course, City, Marks
FROM OnlineEnrollment
INTERSECT
SELECT StudentID, StudentName, Course, City, Marks
FROM OfflineEnrollment;
```

The screenshot shows the phpMyAdmin interface for a database named 'collegedb'. The selected table is 'OnlineEnrollment'. A SQL query is entered in the query box:

```
SELECT StudentID, StudentName, Course, City, Marks FROM OnlineEnrollment UNION SELECT StudentID, StudentName, Course, City, Marks FROM OfflineEnrollment;
```

The query results are displayed in a table with 4 rows and 5 columns:

| StudentID | StudentName | Course | City   | Marks |
|-----------|-------------|--------|--------|-------|
| 1         | Aarav       | BCA    | Pune   | 85    |
| 2         | Priya       | BSc IT | Mumbai | 78    |
| 3         | Rohan       | BSc CS | Delhi  | 92    |
| 4         | Neha        | BCA    | Pune   | 88    |

The output should contain:

| Column      |
|-------------|
| StudentID   |
| StudentName |
| Course      |
| City        |
| Marks       |

**Q5) Write a MySQL query using EXCEPT to display students who are present only in the online enrollment table.**

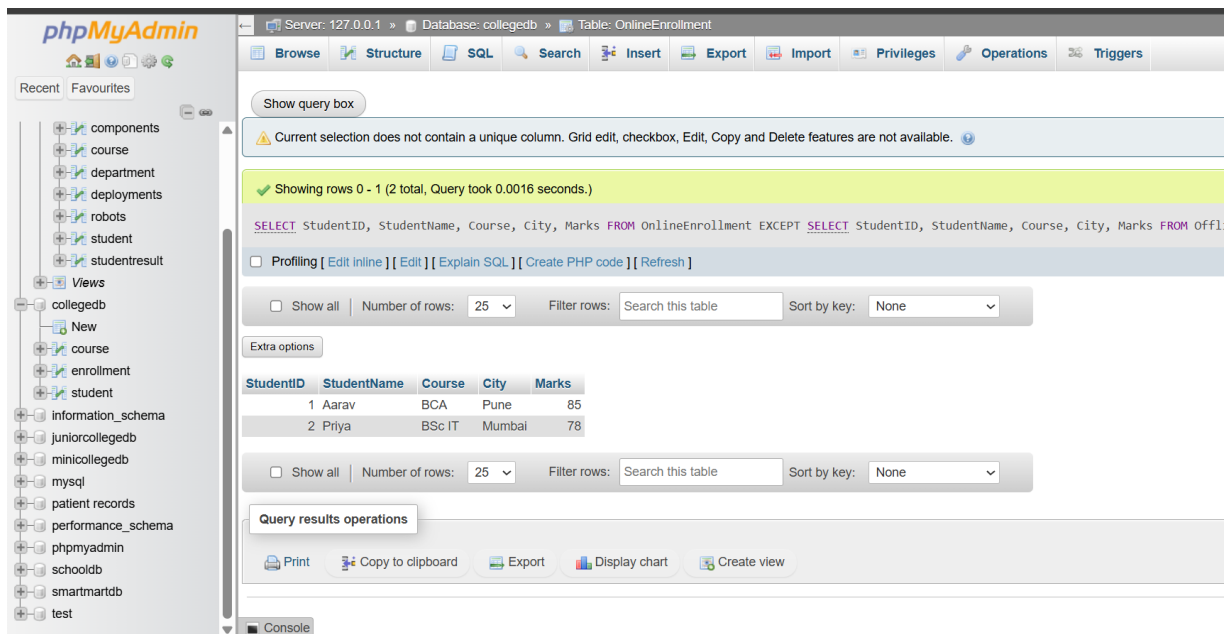
SELECT StudentID, StudentName, Course, City, Marks

FROM OnlineEnrollment

EXCEPT

SELECT StudentID, StudentName, Course, City, Marks

FROM OfflineEnrollment;



The output should contain:

| Column      |
|-------------|
| StudentID   |
| StudentName |
| Course      |
| City        |
| Marks       |

**Q6) Write a MySQL query to display high-scoring students from both tables.**

SELECT StudentID, StudentName, Course, Marks

FROM OnlineEnrollment

WHERE Marks >= 85

UNION

SELECT StudentID, StudentName, Course, Marks

FROM OfflineEnrollment

WHERE Marks >= 85

ORDER BY Marks DESC;

The screenshot shows the phpMyAdmin interface for a database named 'collegedb'. The table 'OnlineEnrollment' is selected. A SQL query is entered in the 'SQL' tab:

```
SELECT StudentID, StudentName, Course, Marks FROM OnlineEnrollment WHERE Marks >= 85 UNION SELECT StudentID, StudentName, Course, Marks FROM OfflineEnrollment WHERE Marks >= 85 ORDER BY Marks DESC;
```

The query results are displayed in a table with 3 rows:

| StudentID | StudentName | Course | Marks |
|-----------|-------------|--------|-------|
| 3         | Rohan       | BSc CS | 92    |
| 4         | Neha        | BCA    | 88    |
| 1         | Aarav       | BCA    | 85    |

The query should:

| Requirement                                            |
|--------------------------------------------------------|
| Select students with marks greater than or equal to 85 |
| Use UNION to combine both tables                       |
| Display StudentID, StudentName, Course, and Marks      |
| Sort the final output by Marks in descending order     |

## Assignment Evaluation

- 0: Not Done [ ]
- 1: Incomplete [ ]
- 2: Late Complete [ ]
- 3: Needs Improvement [ ]
- 4: Complete [ ]
- 5: Well Done [ ]

Signature of Instructor: \_\_\_\_\_

